

11. Ficheros de texto y paso de parámetros por línea de comandos

Mediante un programa en Java se puede acceder al contenido de un fichero grabado en un dispositivo de almacenamiento (por ejemplo en el disco duro) tanto para leer como para escribir (grabar) datos.

La información contenida en las variables, los arrays, los objetos o cualquier otra estructura de datos es volátil, es decir, se pierde cuando se cierra el programa. Los ficheros permiten tener ciertos datos almacenados y disponibles en cualquier momento para cuando nuestro programa los necesite.

Pensemos por ejemplo en un juego. Podríamos utilizar un fichero para guardar el *ranking* de jugadores con las mejores puntuaciones. De esta manera, estos datos no se perderían al salir del juego. De igual forma, en un programa de gestión, puede ser útil tener un fichero de configuración con datos como el número máximo de elementos que se muestran en un listado o los distintos tipos de IVA aplicables a las facturas. Al modificar esta información, quedará almacenada en el fichero correspondiente y no se perderá cuando se cierre el programa.



Cuando un programa se cierra, se pierde la información almacenada en variables, arrays, objetos o cualquier otra estructura. Si queremos conservar ciertos datos, debemos guardarlos en ficheros.

Hay programas que hacen un uso intensivo de datos. Por ejemplo, la aplicación **Séneca** - que, por cierto, está hecha en Java - lleva el control de la matriculación de cientos de miles de alumnos y la gestión de decenas de miles de profesores. En este caso y en otros similares se utiliza un sistema gestor de bases de datos. El acceso a bases de datos mediante Java lo estudiaremos en el [capítulo 13](#).



La creación y uso de ficheros desde un programa en Java se lleva a cabo cuando hay poca información que almacenar o cuando esa información es heterogénea. En los casos en que la información es abundante y homogénea es preferible usar una base de datos relacional (por ejemplo MySQL) en lugar de ficheros.

En este capítulo estudiaremos, además del acceso a ficheros desde un programa en Java, el paso de parámetros desde la línea de comandos. Son cosas diferentes y no hay que utilizarlas juntas necesariamente pero, como podrás comprobar, se combinan muy bien, por eso hemos incluido estos dos recursos de Java en un mismo capítulo.

11.1 Lectura de un fichero de texto

Aunque Java puede manejar también ficheros binarios, vamos a centrarnos exclusivamente en la utilización de ficheros de texto. Los ficheros de texto tienen una gran ventaja, se pueden crear y manipular mediante cualquier editor como por ejemplo **GEdit**.

Siempre que vayamos a usar ficheros, deberemos incluir al principio del programa una o varias líneas para cargar las clases necesarias. Aunque se pueden cargar varias clases en una sola línea usando el asterisco de la siguiente manera:

```
import java.io.*;
```

nosotros no lo haremos así, ya que nuestro código sigue las normas del estándar de Google y estas normas lo prohíben taxativamente. Se importarán las clases usando varias líneas, una línea por cada clase que se importa en el programa; de esta forma:

```
import java.io.BufferedReader;
import java.io.FileReader;
```

No es necesario saber de memoria los nombres de las clases, el entorno de desarrollo **Netbeans** detecta qué clases hacen falta cargar y añade los `import` de forma automática.

Todas las operaciones que se realicen sobre ficheros deberán estar incluidas en un bloque `try-catch`. Esto nos permitirá mostrar mensajes de error y terminar el programa de una forma ordenada en caso de que se produzca algún fallo - el fichero no existe, no tenemos permiso para acceder a él, etc.

El bloque `try-catch` tiene el siguiente formato:

```
try {

    Operaciones_con_fichero

} catch (tipo_de_error nombre_de_variable) {

    Mensaje_de_error

}
```

En el [capítulo 14 sobre control de excepciones](#) se explica de forma detallada el funcionamiento del bloque `try-catch`.

Tanto para leer como para escribir utilizamos lo que en programación se llama un “manejador de fichero”. Es algo así como una variable que hace referencia al fichero con el que queremos trabajar.

En nuestro ejemplo, el manejador de fichero se llama `bf` y se crea de la siguiente manera:

```
BufferedReader bf = new BufferedReader(new FileReader("malaga.txt"));
```

El fichero con el que vamos a trabajar tiene por nombre `malaga.txt` y contiene información extraída de la Wikipedia sobre la bonita ciudad de Málaga¹. A partir de aquí, siempre que queramos realizar una operación sobre ese archivo, utilizaremos su manejador de fichero asociado, es decir `bf`.

En el ejemplo que se muestra a continuación, el programa lee el contenido del fichero `malaga.txt` y lo muestra tal cual en pantalla, igual que si hiciéramos en una ventana de terminal `cat malaga.txt`.

Observa que se va leyendo el fichero línea a línea mediante `bf.readLine()` hasta que se acaban las líneas. Cuando no quedan más líneas por leer se devuelve el valor `null`.

```
import java.io.BufferedReader;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;

/**
 * Ejemplo de uso de la clase File
 * Lectura de un fichero de texto
 *
 * @author Luis José Sánchez
 */
class EjemploFichero01 {
    public static void main(String[] args) {

        try {
            BufferedReader br = new BufferedReader(new FileReader("malaga.txt"));

            String linea = "";

            while (linea != null) {
                System.out.println(linea);
                linea = br.readLine();
            }
        }
    }
}
```

¹<http://es.wikipedia.org/wiki/M%C3%A1laga>

```

        br.close();

    } catch (FileNotFoundException fnfe) { // qué hacer si no se encuentra el fichero

        System.out.println("No se encuentra el fichero malaga.txt");

    } catch (IOException ioe) { // qué hacer si hay un error en la lectura del fichero

        System.out.println("No se puede leer el fichero malaga.txt");

    }
}
}

```

Es importante “cerrar el fichero” cuando se han realizado todas las operaciones necesarias sobre él. En este ejemplo, esta acción se ha llevado a cabo con la sentencia `bf.close()`.

A continuación se muestra un programa un poco más complejo. Se trata de una aplicación que pide por teclado un nombre de fichero. Previamente en ese fichero (por ejemplo `numeros.txt`) habremos introducido una serie de números, a razón de uno por línea. Se podrían leer también los números si estuvieran separados por comas o espacios aunque sería un poco más complicado (no mucho más). Los números pueden contener decimales ya que se van a leer como `Double`. Cada número que se lee del fichero se va sumando de tal forma que la suma total estará contenida en la variable `suma`; a la par se va llevando la cuenta de los elementos que se van leyendo en la variable `i`. Finalmente, dividiendo la suma total entre el número de elementos obtenemos la media aritmética de los números contenidos en el fichero.

```

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

/**
 * Ejemplo de uso de la clase File
 * Calcula la media de los números que se encuentran en un fichero de texto
 *
 * @author Luis José Sánchez
 */
class EjemploFichero08 {
    public static void main(String[] args) {

        System.out.print("Introduzca el nombre del archivo donde se encuentran los números: ");
        String nombreFichero = System.console().readLine();

        try {

```

```
BufferedReader br = new BufferedReader(new FileReader(nombreFichero));

String linea = "";
int i = 0;
double suma = 0;

while (linea != null) {
    i++;
    suma += Double.parseDouble(linea);
    linea = br.readLine();
}
i--;

br.close();

System.out.println("La media es " + suma / (double)i);

} catch (IOException ioe) {
    System.out.println(ioe.getMessage());
}
}
```

11.2 Escritura sobre un fichero de texto

La escritura en un fichero de texto es, si cabe, más fácil que la lectura. Solo hay que cambiar `System.out.print("texto")` por `manejador.write("texto")`. Se pueden incluir saltos de línea, tabuladores y espacios igual que al mostrar un mensaje por pantalla.

Es importante ejecutar `close()` después de realizar la escritura; de esta manera nos aseguramos que se graba toda la información en el disco.

Al realizar escrituras en ficheros con Java hay que tener ciertas precauciones. Cuando toca dar este tema en clase es frecuente que a más de un alumno le empiece a ir lento el ordenador, luego se le queda inutilizado y, por último, ni siquiera le arranca ¿qué ha pasado? Pues que ha estado escribiendo datos en ficheros y por alguna razón, su programa se ha metido en un bucle infinito lo que da como resultado cuelgues y ficheros de varios gigabytes de basura. Por eso es muy importante asegurarse bien de que la información que se va a enviar a un fichero es la correcta ¿cómo? muy fácil, enviándola primero a la pantalla.



Primero a pantalla y luego a fichero

Envía primero a la pantalla todo lo que quieras escribir en el fichero. Cuando compruebes que lo que se ve por pantalla es realmente lo que quieres grabar en el fichero, entonces y solo entonces, cambia `System.out.print("texto")` por `manejador.write("texto")`.

A continuación se muestra un programa de ejemplo que crea un fichero de texto y luego escribe en él tres palabras, una por cada línea.

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

/**
 * Ejemplo de uso de la clase File
 * Escritura en un fichero de texto
 *
 * @author Luis José Sánchez
 */
class EjemploFichero02 {
    public static void main(String[] args) {

        try {
            BufferedWriter bw = new BufferedWriter(new FileWriter("fruta.txt"));

            bw.write("naranja\n");
            bw.write("mango\n");
            bw.write("chirimoya\n");

            bw.close();

        } catch (IOException ioe) {
            System.out.println("No se ha podido escribir en el fichero");
        }
    }
}
```

11.3 Lectura y escritura combinadas

Las operaciones de lectura y escritura sobre ficheros se pueden combinar de tal forma que haya un flujo de lectura y otro de escritura, uno de lectura y dos de escritura, tres de lectura, etc.

En el ejemplo que presentamos a continuación hay dos flujos de lectura y uno de escritura. Observa que se declaran en total tres manejadores de fichero (dos para lectura y uno para escritura). El programa va leyendo, de forma alterna, una línea de cada fichero - una línea de `fichero1.txt` y otra línea de `fichero2.txt` - mientras queden líneas por leer en alguno de los ficheros; y al mismo tiempo va guardando esas líneas en otro fichero con nombre `mezcla.txt`.

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

/**
 * Ejemplo de uso de la clase File
 * Mezcla de dos ficheros
 *
 * @author Luis José Sánchez
 */
class EjemploFichero03 {
    public static void main(String[] args) {

        try {
            BufferedReader br1 = new BufferedReader(new FileReader("fichero1.txt"));
            BufferedReader br2 = new BufferedReader(new FileReader("fichero2.txt"));
            BufferedWriter bw = new BufferedWriter(new FileWriter("mezcla.txt"));

            String linea1 = "";
            String linea2 = "";

            while ( (linea1 != null) || (linea2 != null) ) {
                linea1 = br1.readLine();
                linea2 = br2.readLine();
                if (linea1 != null) {
                    bw.write(linea1 + "\n");
                }
                if (linea2 != null) {
                    bw.write(linea2 + "\n");
                }
            }

            br1.close();
            br2.close();
            bw.close();

            System.out.println("Archivo mezcla.txt creado satisfactoriamente.");
        }
    }
}
```

```
    } catch (IOException ioe) {  
        System.out.println("Se ha producido un error de lectura/escritura");  
        System.err.println(ioe.getMessage());  
    }  
}  
}
```

11.4 Otras operaciones sobre ficheros

Además de leer desde o escribir en un fichero, hay otras operaciones relacionadas con los archivos que se pueden realizar desde un programa escrito en Java.

Veremos tan solo un par de ejemplos. Si quieres profundizar más, échale un vistazo a la [documentación oficial de la clase File](#)².

El siguiente ejemplo muestra por pantalla un listado con todos los archivos que contiene un directorio.

```
import java.io.File;  
  
/**  
 * Ejemplo de uso de la clase File  
 * Listado de los archivos del directorio actual  
 *  
 * @author Luis José Sánchez  
 */  
class EjemploFichero04 {  
    public static void main(String[] args) {  
  
        File f = new File("."); // se indica la ruta entre comillas  
                                // el punto (.) es el directorio actual  
  
        String[] listaArchivos = f.list();  
        for(String nombreArchivo : listaArchivos) {  
            System.out.println(nombreArchivo);  
        }  
    }  
}
```

El siguiente programa de ejemplo comprueba si un determinado archivo existe o no mediante `exists()` y, en caso de que exista, lo elimina mediante `delete()`. Si intentáramos borrar un archivo que no existe obtendríamos un error.

²<http://docs.oracle.com/javase/7/docs/api/java/io/File.html>

```
import java.io.File;

/**
 * Ejemplo de uso de la clase File
 * Comprobación de existencia y borrado de un fichero
 *
 * @author Luis José Sánchez
 */
class EjemploFichero05 {
    public static void main(String[] args) {

        System.out.print("Introduzca el nombre del archivo que desea borrar: ");
        String nombreFichero = System.console().readLine();

        File fichero = new File(nombreFichero);

        if (fichero.exists()) {
            fichero.delete();
            System.out.println("El fichero se ha borrado correctamente.");
        } else {
            System.out.println("El fichero " + nombreFichero + " no existe.");
        }
    }
}
```

11.5 Paso de argumentos por línea de comandos

Como dijimos al comienzo de este capítulo, el paso de argumentos por línea de comandos no está directamente relacionado con los ficheros, aunque es muy frecuente combinar estos dos recursos.

Seguro que has usado muchas veces el paso de argumentos por línea de comandos sin saberlo. Por ejemplo, si tecleas el siguiente comando en una ventana de terminal:

```
head -5 /etc/bash.bashrc
```

se muestran por pantalla las 5 primeras líneas del fichero `bash.bashrc` que se encuentra en el directorio `/etc`. En este caso, `head` es el nombre del programa que estamos ejecutando y los valores `-5` y `/etc/bash.bashrc` son los argumentos.

Volvamos al comienzo, al primer capítulo. ¿Recuerdas cómo ejecutaste tu primer programa en Java? ¡Qué tiempos aquéllos! El programa `HolaMundo` se ejecutaba así:

```
java HolaMundo
```

Para probar los ejemplos que te presentamos más adelante, deberás hacer lo mismo y, además, añadir a continuación y separados por espacios los argumentos que quieres que lea el programa. Vamos a verlo en detalle.

El siguiente programa de ejemplo simplemente muestra por pantalla los argumentos introducidos. Compila el programa y prueba a ejecutar lo siguiente:

```
java EjemploArgumentos06 hola que tal 24 1.2 fin
```

con lo que obtendrás el siguiente resultado:

```
Los argumentos introducidos son:
hola
que
tal
24
1.2
fin
```

Los argumentos han pasado al programa en virtud del parámetro que incluimos en la línea del `main`:

```
public static void main(String[] args) {
```

Fíjate en lo que hay entre paréntesis: `String[] args`. Se trata de un array de cadenas de caracteres (`String`) donde cada uno de los elementos será un argumento que se ha pasado como parámetro. Si has ejecutado el ejemplo tal y como se ha indicado, `args[0]` vale “hola”, `args[1]` vale “que”, `args[2]` vale “tal”, `args[3]` vale “24”, `args[4]` vale “1.2” y `args[5]` vale “fin”.

```
/**
 * Paso de argumentos en la línea de comandos
 *
 * @author Luis José Sánchez
 */
class EjemploArgumentos06 {
    public static void main(String[] args) {
        System.out.println("Los argumentos introducidos son: ");
        for (int i = 0; i < args.length; i++) {
            System.out.println(args[i]);
        }
    }
}
```

Hagamos algo un poco más útil con los argumentos. En el siguiente ejemplo, se suman todos los argumentos que se pasan como parámetros y se muestra por pantalla el resultado de la suma.

Después de compilar, deberás ejecutar desde una ventana de terminal lo siguiente (puedes cambiar los números si quieres):

```
java EjemploArgumentos07 10 36 44
```

con lo que obtendrás este resultado:

```
90
```

es decir, la suma de los números que has pasado como parámetros.

```
/**
 * Paso de argumentos en la línea de comandos
 *
 * @author Luis José Sánchez
 */
class EjemploArgumentos07 {
    public static void main(String[] args) {
        int suma = 0;

        for (int i = 0; i < args.length; i++) {
            suma += Integer.parseInt(args[i]);
        }

        System.out.println(suma);
    }
}
```

Observa que el programa convierte en números enteros todos y cada uno de los argumentos mediante el método `Integer.parseInt()` para poder sumarlos.



Los argumentos recogidos por línea de comandos se guardan siempre en un array de `String`. Cuando sea necesario realizar operaciones matemáticas con esos argumentos habría que convertirlos al tipo adecuado mediante `Integer.parseInt()` o `Double.parseDouble()`.

11.6 Combinación de ficheros y paso de argumentos

Llegó el momento de combinar las operaciones con ficheros con el paso de parámetros por línea de comandos. El ejemplo que mostramos a continuación es parecido al que hemos visto en el apartado anterior; calcula la media de una serie de números, pero esta vez esos números se leen desde un fichero y lo que se pasa como parámetro por la línea de comandos es precisamente el nombre del fichero donde están esos números.

Crea un fichero y nómbralo `numeros.txt` e introduce los siguientes números (es importante que estén separados por un salto de línea para que el programa funcione bien):

```
25
100
44
17
6
8
```

A continuación, compila el programa de ejemplo `EjemploFichero09.java` y teclea lo siguiente en la ventana de terminal:

```
java EjemploFichero09 numeros.txt
```

Aparecerá el siguiente resultado:

```
La media es 33.33333333333336
```

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

/**
 * Ejemplo de uso de la clase File
 * Calcula la media de los números que se encuentran en un fichero de texto.
 * El nombre del fichero se pasa como argumento en la línea de comandos.
 *
 * @author Luis José Sánchez
 */
class EjemploFichero09 {
    public static void main(String[] args) {

        if (args.length != 1) {
```

```

        System.out.println("Este programa calcula la media de los números contenidos en un fiche\
ro.");
        System.out.println("Uso del programa: java EjemploFichero09 FICHERO");
        System.exit(-1); // sale del programa
    }

    try {
        BufferedReader bf = new BufferedReader(new FileReader(args[0]));

        String linea = "";
        int i = 0;
        double suma = 0;

        while (linea != null) {
            i++;
            suma += Double.parseDouble(linea);
            linea = bf.readLine();
        }
        i--;

        bf.close();

        System.out.println("La media es " + suma/(double)i);

    } catch (IOException e) {
        System.out.println(e.getMessage());
    }
}

```

Observa que el programa comprueba el número de argumentos que se pasan por la línea de comandos mediante `if (args.length != 1)` y si este número es distinto de 1, muestra este mensaje:

```

Este programa calcula la media de los números contenidos en un fichero.
Uso del programa: java EjemploFichero09 FICHERO

```

y sale del programa. De esta manera el usuario sabe exactamente cómo hay que utilizar el programa y cómo hay que pasarle la información.

11.7 Procesamiento de archivos de texto

La posibilidad de realizar desde Java operaciones con ficheros abre muchas posibilidades a la hora de procesar archivos: cambiar una palabra por otra, eliminar ciertos

caracteres, mover de sitio una línea o una palabra, borrar espacios o tabulaciones al final de las líneas o cualquier otra cosa que se nos pueda ocurrir.

Cuando se procesa un archivo de texto, los pasos a seguir son los siguientes:

1. Leer una línea del fichero origen mientras quedan líneas por leer.
2. Modificar la línea (normalmente utilizando los métodos que ofrece la clase `String`).
3. Grabar la línea modificada en el fichero destino.
4. Volver al paso 1.

A continuación tienes algunos métodos de la clase `String` que pueden resultar muy útiles para procesar archivos de texto:

- **`charAt(int n)`** Devuelve el carácter que está en la posición *n*-ésima de la cadena. Recuerda que la primera posición es la número 0.
- **`indexOf(String palabra)`** Devuelve un número que indica la posición en la que comienza una palabra determinada.
- **`length()`** Devuelve la longitud de la cadena.
- **`replace(char c1, char c2)`** Devuelve una cadena en la que se han cambiado todas las ocurrencias del carácter *c1* por el carácter *c2*.
- **`substring(int inicio, int fin)`** Devuelve una subcadena.
- **`toLowerCase()`** Convierte todas las letras en minúsculas.
- **`toUpperCase()`** Convierte todas las letras en mayúsculas.

Puedes consultar todos los métodos de la clase `String` en la [documentación oficial](http://docs.oracle.com/javase/7/docs/api/java/lang/String.html)³.

A continuación tienes un ejemplo en el que se usan los métodos descritos anteriormente.

```
/**
 * Ejemplos de uso de String
 *
 * @author Luis José Sánchez
 */
public class EjemplosString {
    public static void main(String[] args) {

        System.out.println("\nEjemplo 1");
        System.out.println("En la posición 2 de \"berengena\" está la letra "
            + "berengena".charAt(2));

        System.out.println("\nEjemplo 2");
        String frase = "Hola caracola.";
```

³ <http://docs.oracle.com/javase/7/docs/api/java/lang/String.html>

```
char[] trozo = new char[10];
trozo[0] = 'z'; trozo[1] = 'z'; trozo[2] = 'z';
frase.getChars(2, 7, trozo, 1);
System.out.print("El array de caracteres vale ");
System.out.println(trozo);

System.out.println("\nEjemplo 3");
System.out.println("La secuencia \"co\" aparece en la frase en la posición "
    + frase.indexOf("co"));

System.out.println("\nEjemplo 4");
System.out.println("La palabra \"murciélago\" tiene "
    + "murciélago".length() + " letras");

System.out.println("\nEjemplo 5");
String frase2 = frase.replace('o', 'u');
System.out.println(frase2);

System.out.println("\nEjemplo 6");
frase2 = frase.substring(3, 10);
System.out.println(frase2);

System.out.println("\nEjemplo 7");
System.out.println(frase.toLowerCase());

System.out.println("\nEjemplo 8");
System.out.println(frase.toUpperCase());
}
}
```

A continuación se muestra un programa que procesa archivos de texto. Lo que hace es cambiar cada tabulador por dos espacios en blanco. Lo hice a propósito cuando estaba escribiendo este manual ya que el [estándar de Google para la codificación en Java](http://google-styleguide.googlecode.com/svn/trunk/javaguide.html)⁴ especifica que la sangría debe ser precisamente de dos espacios. Tenía muchos programas escritos en Java (ejemplos y soluciones a ejercicios) que no cumplían este requisito porque la sangría estaba aplicada con el carácter de tabulación; tenía dos posibilidades, abrir todos y cada uno de los ficheros y cambiarlo a mano, o escribir un programa que lo hiciera. Opté por la segunda opción y aquí tienes el programa ¿verdad que es muy sencillo?

⁴<http://google-styleguide.googlecode.com/svn/trunk/javaguide.html>

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

/**
 * Cambia los tabuladores por 2 espacios
 * @author Luis José Sánchez
 */
public class EjemploProcesamiento10 {
    public static void main(String[] args) {

        for (int i = 0; i < args.length; i++) {

            System.out.print("Procesando el archivo " + args[i] + "...");

            try {
                // renombra el fichero añadiendo ".tmp"
                File ficheroOriginal = new File(args[i]);
                File ficheroTemporal = new File(args[i] + ".tmp");
                ficheroOriginal.renameTo(ficheroTemporal);

                // lee los datos del archivo temporal
                BufferedReader bf = new BufferedReader(new FileReader(args[i] + ".tmp"));

                // crea un fichero nuevo con el nombre original
                BufferedWriter bw = new BufferedWriter(new FileWriter(args[i]));

                String linea = "";

                while (linea != null) {
                    linea = bf.readLine();

                    if (linea != null) {
                        // cambia el tabulador por 2 espacios
                        linea = linea.replace("\t", "  ");

                        bw.write(linea + "\n");
                    }
                }

                bf.close();
                bw.close();
            }
        }
    }
}
```

```
        // borra el fichero temporal
        ficheroTemporal.delete();

    } catch (IOException ioe) {
        System.out.println("Se ha producido un error de lectura/escritura");
        System.err.println(ioe.getMessage());
    }

    System.out.println("hecho");
}
}
```

11.8 Ejercicios



Ejercicio 1

Escribe un programa que guarde en un fichero con nombre `primos.dat` los números primos que hay entre 1 y 500.



Ejercicio 2

Realiza un programa que lea el fichero creado en el ejercicio anterior y que muestre los números por pantalla.



Ejercicio 3

Escribe un programa que guarde en un fichero el contenido de otros dos ficheros, de tal forma que en el fichero resultante aparezcan las líneas de los primeros dos ficheros mezcladas, es decir, la primera línea será del primer fichero, la segunda será del segundo fichero, la tercera será la siguiente del primer fichero, etc.

Los nombres de los dos ficheros origen y el nombre del fichero destino se deben pasar como argumentos en la línea de comandos.

Hay que tener en cuenta que los ficheros de donde se van cogiendo las líneas pueden tener tamaños diferentes.



Ejercicio 4

Realiza un programa que sea capaz de ordenar alfabéticamente las palabras contenidas en un fichero de texto. El nombre del fichero que contiene las palabras se debe pasar como argumento en la línea de comandos. El nombre del fichero resultado debe ser el mismo que el original añadiendo la coletilla `sort`, por ejemplo `palabras_sort.txt`. Suponemos que cada palabra ocupa una línea.



Ejercicio 5

Escribe un programa capaz de quitar los comentarios de un programa de Java. Se utilizaría de la siguiente manera:

```
quita_comentarios PROGRAMA_ORIGINAL PROGRAMA_LIMPIO
```

Por ejemplo:

```
quita_comentarios hola.java holav2.java
```

crea un fichero con nombre `holav2.java` que contiene el código de `hola.java` pero sin los comentarios.



Ejercicio 6

Realiza un programa que diga cuántas ocurrencias de una palabra hay en un fichero. Tanto el nombre del fichero como la palabra se deben pasar como argumentos en la línea de comandos.